Coding Assignment: Embedded Linux IoT Telemetry Reporter

Background

Our device is an embedded Linux telematics device. It collects telemetry from local sensors/services and reports data to the cloud using MQTT. The device may experience unreliable network connectivity, so the reporting service must handle disconnects, reconnects, and temporary local buffering.

For this assignment, please implement a small prototype of a device-side telemetry reporting service.

The goal is not to build a large production system. The goal is to demonstrate your practical hands-on ability in embedded software, Linux service design, C/C++ or Python implementation, MQTT, error handling, and documentation.

Main Requirements

Please build a command-line application called:

```
edgenode-telemetry-reporter
```

The application should simulate collecting telemetry from a local source and publishing it to an MQTT broker.

You may implement it in either **C, C++, or Python**. However, if you choose Python, please also include one small C or C++ component, such as a parser, queue, or helper library, to demonstrate your C/C++ hands-on ability.

Functional Requirements

1. Telemetry Input

The application should read simulated sensor data from a local text file.

Example input file:

```
TEMP=23.5,HUM=61.2,VOLT=12.4
TEMP=23.6,HUM=61.1,VOLT=12.3
```

```
TEMP=bad,HUM=60.9,VOLT=12.2
TEMP=23.7,VOLT=12.1
```

The program should parse each line and create a telemetry message.

Each valid telemetry message should include:

```
{
  "deviceId": "edgenode-test-001",
  "timestamp": "2026-06-30T12:00:00Z",
  "seq": 1,
  "schema": "telemetry.v1",
  "data": {
    "temperature": 23.5,
    "humidity": 61.2,
    "voltage": 12.4
  }
}
```

2. Input Validation

The parser should handle:

- Missing fields.
- Invalid numeric values.
- Extra unknown fields.
- Empty lines.
- Lines with incorrect format.

The program should not crash on bad input. It should log useful error messages and continue processing the next line.

3. Local Buffering

The application should maintain a local queue of messages before publishing.

The queue may be implemented as:

- An in-memory ring buffer, or
- A persistent SQLite/file-based queue.

A persistent queue is preferred, but not required.

The queue should have a maximum size. When the queue is full, please clearly document your chosen behavior, for example:

- Drop newest message.
- Drop oldest message.
- Stop accepting new messages.
- Persist to disk.

4. MQTT Publishing

The application should publish telemetry messages to an MQTT broker.

For testing, you may use a local Mosquitto broker running on the same machine.

Example topic:

```
company/edgenode/edgenode-test-001/telemetry
```

The MQTT broker address, port, device ID, and topic prefix should be configurable.

Example config file:

```
device_id=edgenode-test-001
mqtt_host=localhost
mqtt_port=1883
topic_prefix=company/edgenode
input_file=telemetry_input.txt
publish_interval_ms=1000
queue_max_size=100
```

5. Network Outage Handling

The program should handle MQTT connection failure gracefully.

Please demonstrate the following behavior:

- If MQTT broker is unavailable, messages should remain queued.
- The application should retry connection with backoff.
- Once the broker becomes available again, queued messages should be published.
- The application should not crash or busy-loop while disconnected.

6. Logging

The application should log important events, such as:

- Service startup.
- Config loaded.
- Input line parsed successfully.
- Invalid input line rejected.
- MQTT connected/disconnected.
- Publish success/failure.
- Queue size.
- Dropped message, if applicable.

Logging to console is acceptable. Bonus points for syslog/journald-style logging.

7. Graceful Shutdown

The application should handle `Ctrl+C` or SIGTERM gracefully.

Before exiting, it should:

- Stop reading new input.
- Finish or safely preserve queued messages.
- Disconnect from MQTT cleanly.
- Print a final summary.

8. Documentation

Please provide a `README.md` that explains:

- How to build the project.
- How to run the project.
- How to run a local MQTT broker for testing.
- Example input file.
- Example output.
- How network outage behavior can be tested.
- Design decisions and tradeoffs.
- Known limitations.

Optional Bonus Requirements

These are not mandatory, but they are useful if time permits.

Bonus 1: systemd Service

Provide a sample `systemd` service file:

```
[Unit]
Description=EdgeNode Telemetry Reporter
After=network-online.target
Wants=network-online.target

[Service]
ExecStart=/usr/local/bin/edgenode-telemetry-reporter -c /etc/edgenode/
reporter.conf
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Bonus 2: Unit Tests

Add basic tests for:

- Telemetry parser.
- Queue behavior.
- Invalid input handling.

Bonus 3: Docker Test Environment

Provide a simple `docker-compose.yml` that starts a Mosquitto broker and allows the reporter to be tested locally.

Bonus 4: AWS IoT Design Note

Add a short section in the README explaining what would need to change to connect this prototype to AWS IoT Core using MQTT over TLS with device certificates.

Expected Deliverables

Please submit:

1. Source code.
2. Build/run instructions.
3. Sample config file.
4. Sample telemetry input file.
5. README.md.
6. Any tests, if implemented.
7. Optional: systemd service file.
8. Optional: Docker/docker-compose setup.

Please also include a short design note explaining:

- Why you chose your programming language.
- How your queue works.
- How reconnect/backoff works.
- How you would improve this for production use on a Yocto-based embedded Linux device.

Time Expectation

This assignment is expected to take approximately **1–3 days of focused work**.

We are not looking for a perfect production product. We are looking for clean, practical, maintainable code and good engineering judgment.